MATH 345 Unit 1: Coding summary

Setup, arrays, and shapes

Import: `import numpy as np` makes NumPy available as `np`. An assignment such as `u = ...` stores a value. A comment begins with `#`; a notebook displays the last expression in a cell.

```
import numpy as np
u = np.array([3., 4.])
A = np.array([[1., 2.], [0., 1.]])
```

Construct arrays: `np.array([...])` creates a one-dimensional array; a list of row lists creates a matrix array. `3.` means `3.0`. `np.array([[1, 2], [0, 1]], dtype=float)` explicitly uses floating-point entries.

Read dimensions: `u.shape` is `(2,)`; `A.shape` is `(2, 2)`. For a matrix, the pair means (rows, columns). A shape `(n,)` is a one-dimensional array, not an $n \times 1$ matrix.

Create ones: `np.ones((m, n))` creates an $m \times n$ array of ones; the inner parentheses supply its shape.

Arithmetic and products

Entrywise arithmetic: `u + v`, `u - v`, `2 * u`, and `u / 2` add, subtract, scale, or divide entries. For same-shaped arrays, `u * v` multiplies corresponding entries.

Dot product: `u @ v` returns $\mathbf{u} \cdot \mathbf{v}$, a scalar, when both arrays are one-dimensional and have the same length.

Matrix-vector product: `A @ u` computes $A\mathbf{u}$. Shapes `(m, n)` and `(n,)` give an output of shape `(m,)`.

Matrix product: `A @ B` computes AB. Shapes `(m, n)` and `(n, p)` give `(m, p)`. For NumPy arrays, use `@` for these products; `*` is entrywise.

Transpose: `A.T` swaps rows and columns, changing `(m, n)` to `(n, m)`. Transposing a one-dimensional array leaves its shape `(n,)` unchanged.

Lengths, distances, and similarity

Norm: `np.linalg.norm(u)` returns the Euclidean length $\|\mathbf{u}\|$. `np.linalg.norm(u - v)` returns the distance between two vectors.

Cosine similarity: for nonzero vectors of the same length,

```
cosine = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
```

This is $\mathbf{u} \cdot \mathbf{v} / (\|\mathbf{u}\| \, \|\mathbf{v}\|)$. Larger cosine means closer directions; smaller distance means closer points.

Row-wise scores and rankings

One norm per row: `np.linalg.norm(X, axis=1)` computes along each row. If `X` has shape `(m, n)`, the result has shape `(m,)`. For nonzero document rows and a nonzero query `q` of shape `(n,)`,

```
scores = (X @ q) / (
    np.linalg.norm(X, axis=1) * np.linalg.norm(q)
)
```

`X @ q` gives one dot product per row; the division normalizes each score.

Indexing: positions start at zero. `doc_names[0]` selects the first name, `doc_names[1]` the second, and `doc_names[-1]` the last. An array can also contain strings, as in `np.array(["D1", "D2", "D3"])`.

Sorting indices: `np.argsort(scores)` returns the positions of scores from smallest to largest, not the sorted score values. `[::-1]` reverses the order: it is a slice with step −1.

```
order = np.argsort(scores)[::-1]
ranking = doc_names[order]
```

`doc_names[order]` selects the names in decreasing-score order. Keep names and scores in the same original row order.

Weighted combinations

Sum and normalize: `s.sum()` adds all entries of `s`. When scores are nonnegative and their sum is positive, division by the sum produces nonnegative weights that add to 1:

```
s = K @ q
alpha = s / s.sum()
out = alpha @ V
```

With shapes `K`: `(m, d)`, `q`: `(d,)`, and `V`: `(m, p)`, both `s` and `alpha` have shape `(m,)`; `out` has shape `(p,)` and is a weighted average of the rows of `V`. This is the unit's simple weighting rule, not softmax.

Maps, composition, and batches

Define a map: `def` starts a function; the indented `return` line gives its output:

```
def f(x):
    return W @ x + b
```

This represents $f(\mathbf{x}) = W\mathbf{x} + \mathbf{b}$; use `f(x)` to evaluate it.

Composition: `R @ S` applies S first, then R. In general, `R @ S` and `S @ R` differ.

Columns versus rows: if columns of `X` are input vectors, `A @ X` transforms them all. If rows of `Q` and `K_tokens` are query and key vectors, `Q @ K_tokens.T` computes all pairwise dot products. Shapes `(m, d)` and `(n, d)` give a score matrix `(m, n)`.